PROJECT REPORT

MOVIE RECOMMENDER APP

Name: Abhigyan Tiwari

Reg no: 25BAI11270

Branch: CSE (AI-ML)

Course: FUNDAMENTALS IN AI&ML(CSA2001)

INTRODUCTION

In the modern digital era, users are exposed to a vast amount of entertainment content, making it difficult to choose relevant movies. A recommendation system helps users by suggesting movies based on their preferences.

This project implements a **Movie Recommendation System** using **content-based filtering**, where recommendations are generated based on similarities between movies using features like genres, keywords, and overview.

MOTIVATION OF THE PROJECT

- To simplify movie selection for users
- To understand practical implementation of machine learning
- To explore real-world applications of AI
- To build an interactive GUI-based application

PROBLEM STATEMENT

Users often struggle to find movies that match their interests due to the large number of available options. Traditional browsing is inefficient and time-consuming.

This project aims to develop a system that:

- Takes a movie as input
- Suggests similar movies automatically

OBJECTIVE OF THE PROJECT

- To build a movie recommendation system
- To implement content-based filtering
- To use cosine similarity for recommendation
- To create a user-friendly GUI using Tkinter
- To fetch real-time movie details using OMDb API

EXISTING METHODS

1. Content-Based Filtering
2. Collaborative Filtering
3. Hybrid Systems

PROS AND CONS OF THE STATED METHODS

Content-Based Filtering

Pros:

- No need for user data
- Works for new users

Cons:

- Limited diversity
- Depends on dataset quality

Collaborative Filtering

Pros:

- More personalized recommendations

Cons:

- Cold start problem
- Requires user history

Hybrid Systems

Pros:

- More accurate

Cons:

- Complex implementation

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware:

- Minimum 4GB RAM
- Basic computer/laptop

Software:

- Python (3.x)
- Libraries:
  - Pandas
  - Scikit-learn
  - Requests
  - Tkinter
  - PIL
- Dataset: movies.csv
- Internet connection (for API)

METHODOLOGY AND GOAL

Methodology:

1. Load dataset
2. Clean data
3. Combine features
4. Convert text to vectors
5. Calculate similarity
6. Recommend movies
7. Display results via GUI

Goal:

To provide accurate and fast movie recommendations with a simple interface.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

1. **Data Processing Module**
    - Handles dataset loading and cleaning
2. **Feature Extraction Module**
    - Converts text to numerical format
3. **Similarity Module**
    - Computes cosine similarity
4. **Recommendation Module**
    - Suggests top 5 similar movies
5. **API Module**
    - Fetches movie details and posters
6. **GUI Module**
    - User interaction using Tkinter

ALGORITHM DEVELOPMENT

Step-by-step Algorithm:

1. Load dataset using Pandas
2. Fill missing values
3. Combine features (genres + keywords + overview)
4. Apply Count Vectorizer
5. Compute cosine similarity matrix
6. Take user input (movie name)
7. Find index of selected movie
8. Calculate similarity scores
9. Sort and select top 5 movies
10. Displays results
11. Fetch additional details using API

SOFTWARE ARCHITECTURAL DIAGRAM

User Input (GUI)
↓
Movie Selection
↓
Dataset Processing
↓
Feature Extraction (CountVectorizer)
↓
Similarity Calculation (Cosine Similarity)
↓
Recommendation Engine
↓
OMDb API (Poster + Details)
↓
Display Output (GUI)

CODING

The system is implemented in Python using:

- Pandas for dataset
- Scikit-learn for ML
- Tkinter for GUI
- Requests for API

(The full code is included in the project file.)

OUTPUT

- User selects a movie
- System displays:
  - Top 5 recommended movies
  - Movie poster
  - Year, Director, Actors
  - IMDb rating

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

- Content-based recommendation using cosine similarity
- GUI interaction using Tkinter
- API integration for real-time data
- Image display using PIL

SIGNIFICANT PROJECT OUTCOMES

- Successfully built a working recommendation system
- Implemented ML concepts practically
- Created an interactive GUI application
- Integrated external API

TESTING AND REFINEMENT

- Tested with multiple movie inputs
- Handled missing data
- Improved GUI responsiveness
- Fixed API errors and invalid inputs

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

- Netflix, Amazon Prime recommendations
- E-commerce product suggestions
- Music recommendation systems
- Online content platforms

CONTRIBUTION / FINDINGS OF THE PROJECT

- Demonstrated how similarity-based ML works
- Showed importance of feature engineering
- Highlighted real-world use of APIs
- Built a complete AI-based application

LIMITATION / CONSTRAINTS OF THE SYSTEM

- No user personalization
- Depends on dataset quality
- Limited recommendation diversity
- Requires internet for API

CONCLUSION

The project successfully demonstrates a movie recommendation system using machine learning techniques. It provides relevant suggestions based on content similarity and enhances user experience through a graphical interface and real-time data.

FUTURE ENHANCEMENTS

- Add collaborative filtering
- Improve UI design
- Deploy as web app (Streamlit/Flask)
- Add user login system
- Use deep learning models

REFERENCES

- Scikit-learn Documentation
- Pandas Documentation
- OMDb API Documentation
- Python Tkinter Documentation